



Módulo I - Lógica de Programação

Estruturas de Decisão e Repetição
em Algoritmos

0 Que Vamos Aprender Hoje

01

Estruturas de Decisão

SE simples, SE/SENÃO composto e condicionais encadeadas

03

Estruturas de Repetição

ENQUANTO, PARA e REPITA — quando e como usar cada uma

02

Operadores de Condição

Relacionais e lógicos para construir expressões booleanas

04

Exemplo Prático Combinado

Validação de nota com decisão e repetição integradas

Estruturas de Decisão (Condicionais)

As estruturas de decisão permitem que um programa escolha diferentes caminhos de execução com base em condições. Em vez de seguir sempre a mesma sequência de passos, o algoritmo avalia uma expressão e age de acordo com o resultado — **verdadeiro** ou **falso**.

- ❏ Pense assim: toda vez que você toma uma decisão no dia a dia ("SE estiver chovendo, ENTÃO levo o guarda-chuva"), você está executando uma estrutura condicional!



Condiciona1 Simples e Composta

SE Simples

Executa um bloco de código **somente se** a condição for verdadeira. Se for falsa, nada acontece.

```
SE (condição) ENTÃO  
    instrução(ões)  
FIMSE
```

SE/SENÃO Composto

Oferece dois caminhos: um bloco para quando a condição é verdadeira e outro para quando é falsa. **Sempre executa um dos dois blocos.**

```
SE (condição) ENTÃO  
    instrução(ões)  
SENAO  
    instrução(ões)  
FIMSE
```

A estrutura composta é mais completa pois **garante uma resposta independente** do resultado da condição. Use o SE simples apenas quando não há ação a tomar no caso falso.

Operadores para Construção de Condições

Operadores Relacionais

Usados para **comparar** dois valores. O resultado é sempre verdadeiro ou falso.

>

Maior que

<

Menor que

>=

Maior ou igual

<=

Menor ou igual

=

Igual

<>

Diferente

Operadores Lógicos

Usados para **combinar** duas ou mais condições em uma única expressão.

E (AND)

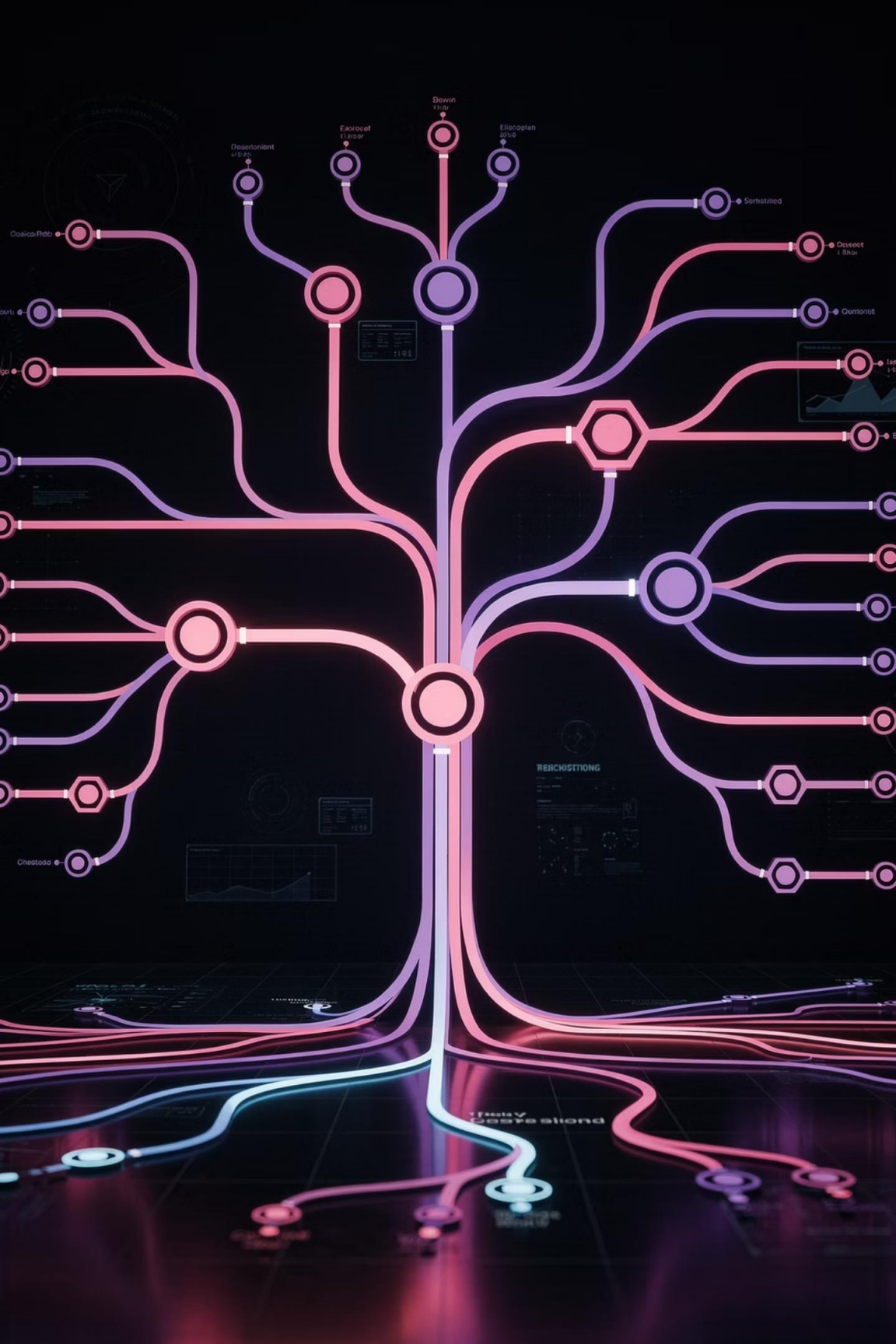
Verdadeiro somente se **todas** as condições forem verdadeiras

OU (OR)

Verdadeiro se **pelo menos uma** condição for verdadeira

NAO (NOT)

Inverte o resultado: verdadeiro vira falso e vice-versa



Condicionais Encadeadas (SE/SENAO SE)

Quando precisamos avaliar **múltiplas condições em sequência**, usamos condicionais encadeadas. O algoritmo testa cada condição em ordem e executa apenas o bloco da primeira condição verdadeira encontrada.

```
SE (condição1) ENTAO  
    instrução(ões)  
SENAO  
    SE (condição2) ENTAO  
        instrução(ões)  
    SENAO  
        instrução(ões)  
FIMSE  
FIMSE
```

- ❏ **A ordem das condições importa! Coloque sempre as condições mais específicas antes das mais genéricas para evitar resultados inesperados.**

Estruturas de Repetição (Laços)

As estruturas de repetição, também chamadas de laços ou loops, permitem executar um bloco de código **múltiplas vezes** sem precisar reescrevê-lo. São fundamentais para processar listas, validar entradas e realizar contagens. Todo laço possui três elementos essenciais:



Condição de Parada

Define quando o laço deve terminar. Sem ela, o programa entra em **loop infinito**.



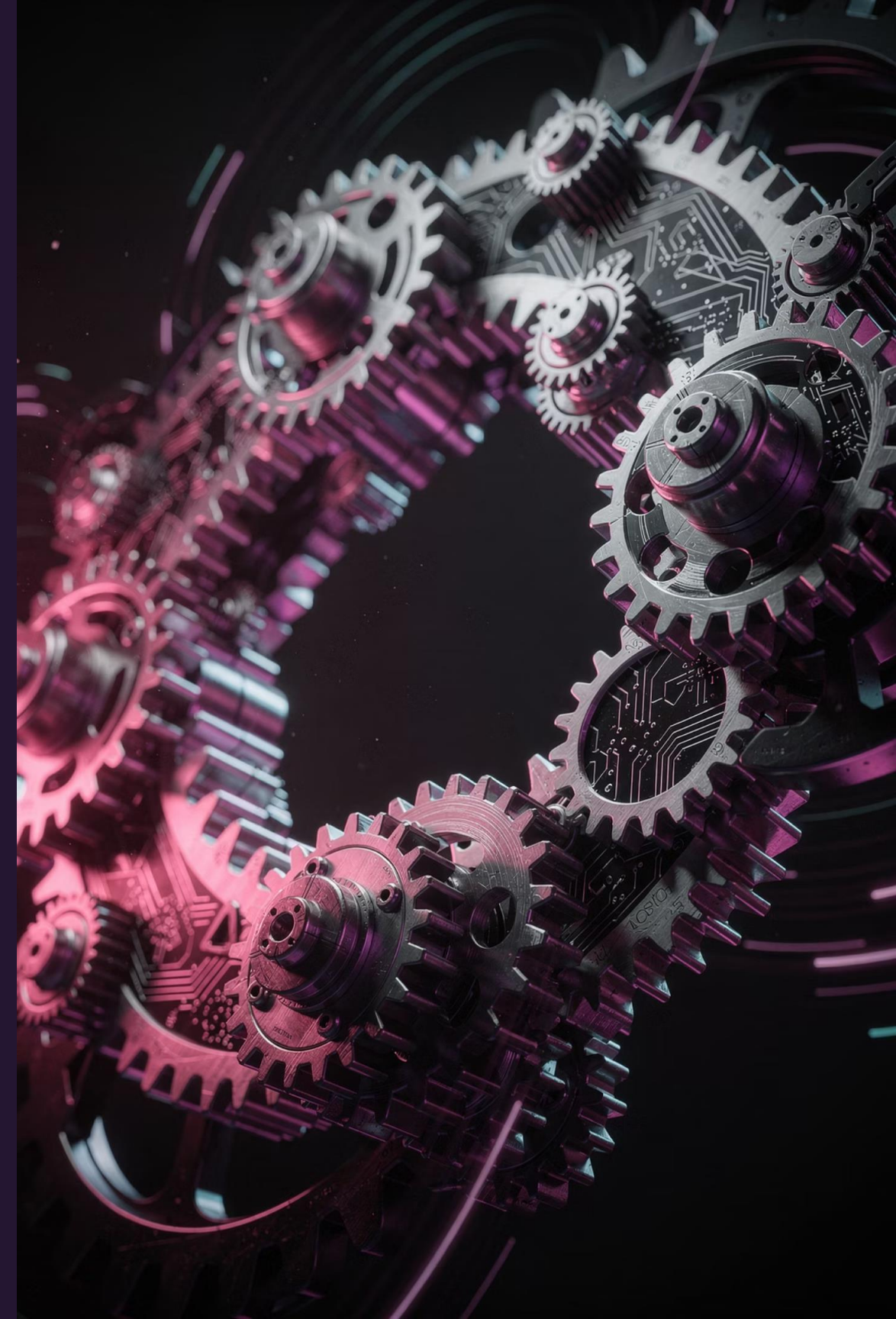
Variável de Controle

Controla o estado do laço e **deve ser atualizada** a cada iteração.



Instruções Internas

O bloco de código que será **repetido** enquanto a condição permitir.



Estrutura ENQUANTO (While)

Como Funciona

Testa a condição antes de executar o bloco. Se a condição for falsa logo de início, o bloco **nunca é executado** — pode rodar 0 ou mais vezes.

```
ENQUANTO (condição) FAÇA  
    instrução(ões)  
FIMENQUANTO
```

Ideal para situações em que o número de repetições é desconhecido previamente.

Exemplo: Contar de 1 a 5

```
contador ← 1  
ENQUANTO (contador <= 5) FAÇA  
    ESCREVA(contador)  
    contador ← contador + 1  
FIMENQUANTO
```

Saída: 1 2 3 4 5

Atenção: a linha `contador ← contador + 1` é obrigatória! Sem ela, o laço nunca terminaria.

Estrutura PARA (For)

Como Funciona

Ideal quando sabemos exatamente quantas repetições serão necessárias. A variável de controle é inicializada, incrementada e testada automaticamente.

```
PARA variável DE valor_inicial ATE valor_final [PASSO n] FACA
    instrução(ões)
FIMPARA
```

O parâmetro opcional PASSO define o incremento. Se omitido, o padrão é +1 a cada iteração.

Exemplo: Tabuada do 5

```
i <- 1
PARA i DE 1 ATE 10 FACA
    ESCREVAL("5 x ", i, " = ", 5*i)
FIMPARA
```

Saída:

5 x 1 = 5

5 x 2 = 10

5 x 3 = 15 ... 5 x 10 = 50

Estrutura REPITA (Do-While)

Como Funciona

Diferente das outras estruturas, o REPITA testa a **condição no final** do bloco. Isso garante que as instruções sejam executadas **pelo menos uma vez**, independente da condição.

```
REPITA
    instrução(ões)
ATE (condição)
```

Perfeito para menus interativos e validação de entradas do usuário.

Exemplo: Menu com Saída

```
REPITA
    ESCREVAL("1 - Opção A")
    ESCREVAL("2 - Opção B")
    ESCREVAL("0 - Sair")
    LEIA(opcao)
ATE (opcao = 0)
```

O menu será exibido sempre ao menos uma vez. O laço continua até o usuário digitar 0, garantindo que ele veja as opções antes de qualquer decisão.

As Três Estruturas de Repetição

Cada estrutura de repetição tem sua **característica principal** que determina quando usá-la. Entender as diferenças é fundamental para escolher a ferramenta certa para cada problema.

Característica	ENQUANTO	PARA	REPITA
Teste da condição	No início	No início	No final
Execução mínima	0 vezes	0 vezes	1 vez
Controle automático	Manual	Automático	Manual
Mais adequado para	Repetições indefinidas	Repetições definidas	Execução garantida

Quando Usar Cada Laço?

Use **ENQUANTO** quando...

Você não sabe quantas vezes o laço vai repetir. Exemplos: ler dados até o usuário digitar "fim", aguardar um evento externo, ou processar um arquivo até o final.

Use **PARA** quando...

O número de repetições é conhecido antes de começar. Exemplos: percorrer todos os elementos de um vetor, gerar tabuadas, ou repetir uma ação exatamente N vezes.

Use **REPITA** quando...

O bloco deve executar pelo menos uma vez antes da verificação. Exemplos: exibir um menu e ler a opção, solicitar uma senha, ou validar entradas do usuário antes de aceitar.

Validação de Nota com Decisão + Repetição

Veja como estruturas de decisão e repetição trabalham juntas na prática. O algoritmo abaixo valida a entrada do usuário e classifica a nota:

```
REPITA
  ESCREVA("Digite a nota (0-10): ")
  LEIA(nota)
ATE (nota >= 0) E (nota <= 10)

SE (nota >= 6) ENTAO
  ESCREVA("Aprovado!")
SENAO
  SE (nota = 5) ENTAO
    ESCREVA("Recuperação!")
  SENAO
    ESCREVA("Reprovado!")
FIMSE
FIMSE
```

nota >= 7
Aprovado

nota >= 5
Recuperação

nota < 5
Reprovado

Por Que Esse Algoritmo Funciona?

O exemplo anterior combina dois conceitos poderosos de forma elegante. Veja o raciocínio por trás de cada escolha:

1 REPITA garante entrada válida

O laço continua pedindo a nota até que o usuário forneça um valor entre 0 e 10. O operador E exige que ambas as condições sejam verdadeiras simultaneamente.

2 SE encadeado classifica corretamente

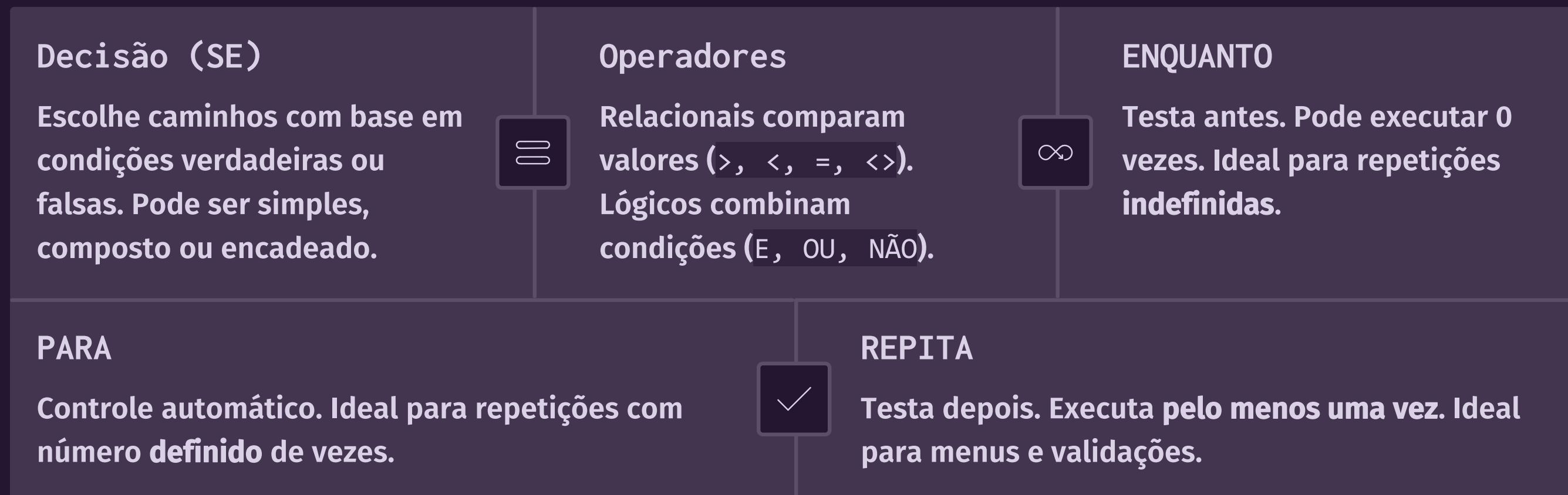
As condições são testadas da mais restritiva (≥ 6) para a menos restritiva ($= 5$). O encadeamento garante que cada nota caia em apenas uma categoria, sem sobreposição.

3 SENAO final captura os demais casos

O último SENAO funciona como um "caso padrão", garantindo que o algoritmo sempre emita uma mensagem, sem deixar nenhuma nota sem resposta.



0 Que Aprendemos Hoje



📌 **Próximo passo:** Pratique criando algoritmos que combinem decisões e laços.